TUGAS M6 - DATA MINING

DECISION TREE

GUSTHI PANGESTU

D4 IT D | 3124600098

1. LOAD DATASET (TRAIN & TEST)

Tugas 5 > jawaban > decision_tree_classifier.py > ...

```python
import os
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier, export_text

# Inisialisasi direktori
base_dir = os.path.dirname(os.path.abspath(__file__))
data_dir = os.path.join(base_dir, '..', 'data')

# 1. Load dataset train, test, dan target
dataset = pd.read_csv(os.path.join(data_dir, 'titanic.csv'))
test_dataset = pd.read_csv(os.path.join(data_dir, 'titanic_test.csv'))
raw_test_label = pd.read_csv(os.path.join(data_dir, 'titanic_testlabel.csv'))

target_col = 'Survived' if 'Survived' in raw_test_label.columns else raw_test_label.columns[1]
test_label = raw_test_label[target_col]
train_label = dataset['Survived']

```

2. FEATURE SELECTION

Tugas 5 > jawaban > decision_tree_classifier.py > ...

```
19
20    # 2. Ambil fitur yang digunakan
21    features = ['Sex', 'Age', 'Pclass', 'Fare']
22    train_data = dataset[features].copy()
23    test_data = test_dataset[features].copy()
24
```

Memilih dan mengambil subset kolom fitur yang relevan, yaitu ***Sex***, ***Age***, ***Pclass***, dan ***Fare***, dari data latih ***train_data*** dan ***test_data*** untuk digunakan dalam proses pemodelan Decision Tree. Penggunaan ***.copy()*** memastikan manipulasi data fitur tidak mengubah DataFrame asli.

3. CATEGORICAL ENCODING

Tugas 5 > jawaban > decision_tree_classifier.py > ...

```
25  # 3. Konversi fitur Sex menjadi numerik (0/1) agar bisa diproses model
26  gender_map = {'male': 0, 'female': 1}
27  train_data['Sex'] = train_data['Sex'].map(gender_map)
28  test_data['Sex'] = test_data['Sex'].map(gender_map)
29
```

Mengonversi fitur kategorikal Sex menjadi nilai numerik male (0) dan female (1) menggunakan mapping. Tahap transformasi ini penting karena algoritma Decision Tree pada Scikit-Learn memerlukan representasi data numerik untuk menentukan splitting threshold.

4. MISSING VALUE IMPUTATION

Tugas 5 > jawaban > decision_tree_classifier.py > ...

```
30  # 4. Imputasi missing value Age & Fare
31  # Isi missing value Age train dengan mean tiap kelas Survived
32  train_data['Age'] = dataset.groupby('Survived')['Age'].transform(
33      lambda x: x.fillna(x.mean())
34  )
35
36  # Isi missing value Age test berdasarkan kelas atau mean data train
37  test_data['Age'] = test_data.groupby(test_label)['Age'].transform(
38      lambda x: x.fillna(x.mean())
39  )
40  test_data['Age'] = test_data['Age'].fillna(train_data['Age'].mean())
41
42  # Isi missing value Fare pada data test dengan nilai mean
43  test_data['Fare'] = test_data['Fare'].fillna(train_data['Fare'].mean())
44
```

Proses imputasi (pengisian missing value) pada fitur Age menggunakan nilai rata-rata (mean) dari masing-masing kelas target Survived sesuai soal nomor 3 dan pengisian nilai kosong pada fitur Fare juga diterapkan pada test_data (soal nomor 4) agar kedua dataset bersih dari nilai NaN dan konsisten saat dievaluasi oleh algoritma Decision Tree.

5. TRAINING MODEL (UNPRUNED VS PRUNED)

Tugas 5 > jawaban > decision_tree_classifier.py > ...

```
45  # 5. Latih model perbandingan (Tanpa Pruning vs Dengan Pruning)
46  # Model 1: Default (Tanpa pembatasan kedalaman)
47  dtc_unpruned = DecisionTreeClassifier(random_state=42)
48  dtc_unpruned.fit(train_data, train_label)
49  acc_unpruned = dtc_unpruned.score(test_data, test_label)
50  err_unpruned = round((1 - acc_unpruned) * 100, 2)
51
52  # Model 2: Disederhanakan (Pre-pruning max_depth=3)
53  dtc = DecisionTreeClassifier(max_depth=3, random_state=42)
54  dtc.fit(train_data, train_label)
55
```

Melatih (fitting) dua variasi model Decision Tree untuk perbandingan, yaitu model unpruned (default tanpa batas kedalaman pohon) dan model pruned menggunakan batasan parameter pre-pruning (max_depth=3) guna mencegah terjadinya overfitting serta mempermudah interpretasi struktur percabangan saat dievaluasi terhadap test_data.

6. PREDIKSI DAN PERHITUNGAN ERROR RATIO

Tugas 5 > jawaban > decision_tree_classifier.py > ...

```
55
56  # 6. Lakukan prediksi & evaluasi pada data test
57  class_result = dtc.predict(test_data)
58  acc = dtc.score(test_data, test_label)
59  err = round((1 - acc) * 100, 2)
60
61  print("=" * 65)
62  print("PERBANDINGAN MODEL DECISION TREE PADA DATA TEST")
63  print("=" * 65)
64  print(f"{'Metrik / Model':<30} | {'Default':>14} | {'max_depth=3':>14}")
65  print("-" * 65)
66  print(f"{'Jumlah Total Node':<30} | {f'{dtc_unpruned.tree_.node_count} node':>14} | {f'{dtc.tree_.node_count} node':>14}")
67  print(f"{'Kedalaman Pohon (Depth)':<30} | {f'{dtc_unpruned.tree_.max_depth} level':>14} | {f'{dtc.tree_.max_depth} level':>14}")
68  print(f"{'Akurasi Pengujian (Accuracy)':<30} | {f'{round(acc_unpruned * 100, 2)}%':>14} | {f'{round(acc * 100, 2)}%':>14}")
69  print(f"{'Error Ratio':<30} | {f'{err_unpruned}%':>14} | {f'{err}%':>14}")
70  print("=" * 65)
71  print(f"Hasil Prediksi 10 sampel pertama (Pruned): {class_result[:10]}\n")
72
```

Pengujian data uji menggunakan fungsi ***predict()*** serta menghitung persentase rasio kesalahan (error ratio) dari nilai akurasi untuk menjawab soal nomor 7. Komparasi antara model default (unpruned) dan model pruned guna membandingkan performa akurasi serta kompleksitas ukuran pohon keputusannya.

6. HASIL PREDIKSI

```
============================================================
PERBANDINGAN MODEL DECISION TREE PADA DATA TEST
============================================================
Metrik / Model                    |        Default |    max_depth=3
------------------------------------------------------------
Jumlah Total Node                 |       381 node |        15 node
Kedalaman Pohon (Depth)           |       19 level |        3 level
Akurasi Pengujian (Accuracy)      |         77.03% |         90.43%
Error Ratio                       |         22.97% |          9.57%
============================================================
Hasil Prediksi 10 sampel pertama (Pruned): [0 0 0 0 1 0 0 0 1 0]
```

7. PRINT TREE STRUCTURE & FEATURE

Tugas 5 > jawaban > decision_tree_classifier.py > ...

```
73  # 7. Informasi Ringkasan Node Pohon Keputusan
74  print("=" * 65)
75  print("INFORMASI STRUKTUR NODE POHON")
76  print("=" * 65)
77  print(f"{'Total Jumlah Node':<30}: {dtc.tree_.node_count} node")
78  print(f"{'Kedalaman Pohon (Depth)':<30}: {dtc.tree_.max_depth} level")
79  print(f"{'Jumlah Leaf Node (Daun)':<30}: {dtc.tree_.n_leaves} daun")
80  print("\nTingkat Kepentingan Fitur (Feature Importances):")
81  for feat, imp in zip(features, dtc.feature_importances_):
82      print(f"  - {feat:<8}: {imp:.4f} ({imp*100:>5.2f}%)")
83
```

7. HASIL PRINT TREE STRUCTURE & FEATURE

```
================================================
INFORMASI STRUKTUR NODE POHON
================================================
Total Jumlah Node                 : 15 node
Kedalaman Pohon (Depth)           : 3 level
Jumlah Leaf Node (Daun)           : 8 daun

Tingkat Kepentingan Fitur (Feature Importances):
  - Sex      : 0.6373 (63.73%)
  - Age      : 0.1268 (12.68%)
  - Pclass   : 0.2359 (23.59%)
  - Fare     : 0.0000 ( 0.00%)
```

8. EXPORT TREE RULES AS TEXT

Tugas 5 > jawaban > decision_tree_classifier.py > ...

```
83
84    # 8. Tampilkan Struktur Node
85    print("\n" + "=" * 65)
86    print("STRUKTUR NODE")
87    print("=" * 65)
88    tree_rules = export_text(dtc, feature_names=features)
89    print(tree_rules)
90
```

8. HASIL EXPORT TREE RULES AS TEXT

```
==============================
STRUKTUR NODE
==============================
|--- Sex <= 0.50
|   |--- Age <= 6.50
|   |   |--- Pclass <= 2.50
|   |   |   |--- class: 1
|   |   |--- Pclass >  2.50
|   |   |   |--- class: 0
|   |--- Age >  6.50
|   |   |--- Pclass <= 1.50
|   |   |   |--- class: 0
|   |   |--- Pclass >  1.50
|   |   |   |--- class: 0
|--- Sex >  0.50
|   |--- Pclass <= 2.50
|   |   |--- Age <= 2.50
|   |   |   |--- class: 0
|   |   |--- Age >  2.50
|   |   |   |--- class: 1
|   |--- Pclass >  2.50
|   |   |--- Age <= 28.67
|   |   |   |--- class: 1
|   |   |--- Age >  28.67
|   |   |   |--- class: 0
```

Struktur aturan logika hasil export_text() dengan batasan max_depth=3 :

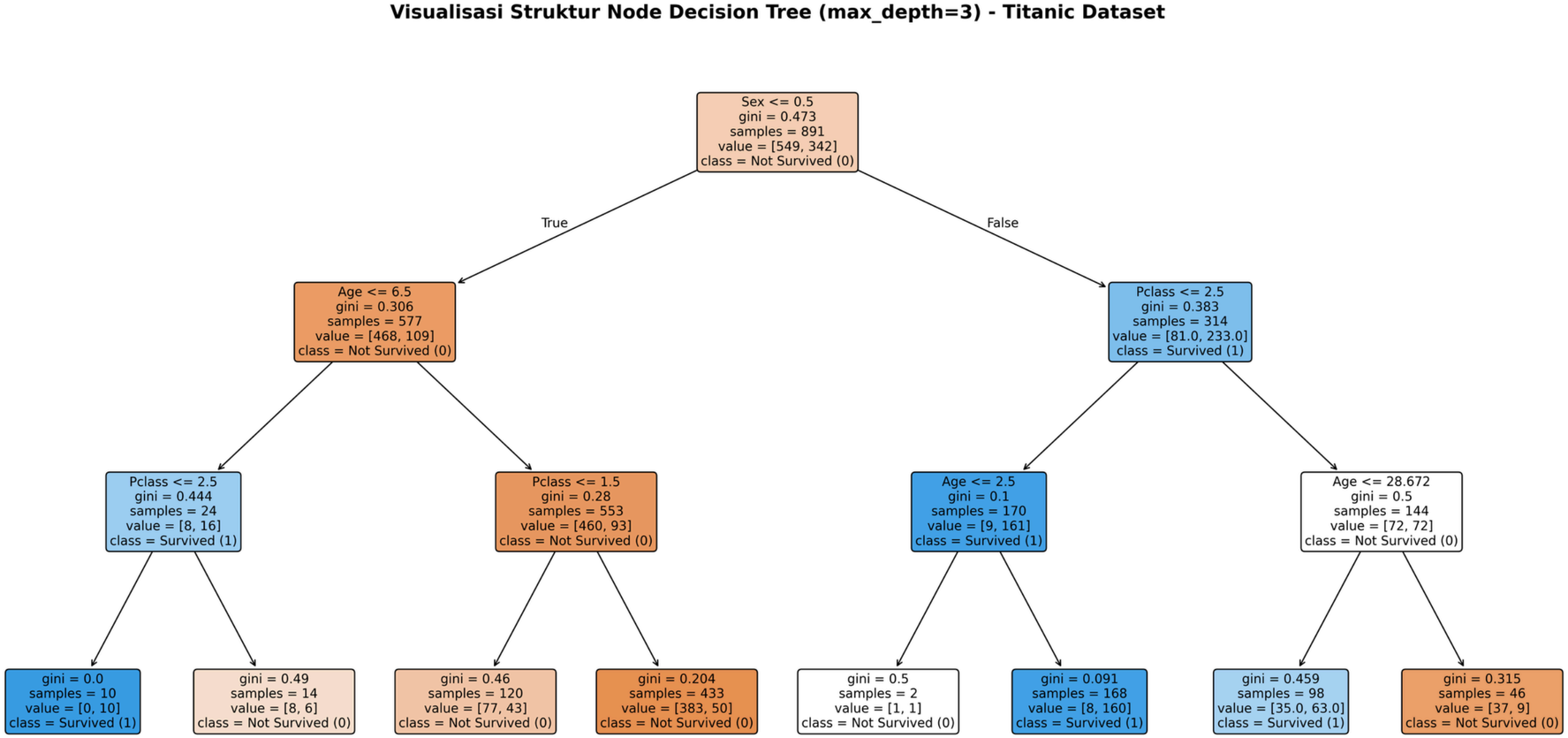
- Akar Keputusan (Sex): Cabang utama langsung memisahkan laki-laki (<= 0.50) dan perempuan (> 0.50) sebagai faktor penentu keselamatan paling dominan.
- Filter Usia & Kelas: Kelompok laki-laki hanya berpeluang selamat jika anak-anak di kelas 1-2, sedangkan perempuan dominan selamat di kelas 1-2 atau usia muda di kelas 3.
- Batas Kedalaman: Percabangan dipangkas tepat di tingkat ke-3 (pre-pruning) agar hasil interpretasi biner (class: 0/1) tetap ringkas.

9. DECISION TREE VISUALIZATION

Tugas 5 > jawaban > decision_tree_classifier.py > ...

```
90
91  # 9. Simpan Visualisasi Gambar Pohon Keputusan (.png via Matplotlib)
92  png_path = os.path.join(base_dir, "decision_tree.png")
93  fig, ax = plt.subplots(figsize=(18, 9), dpi=300)
94  tree.plot_tree(
95      dtc,
96      feature_names=features,
97      class_names=['Not Survived (0)', 'Survived (1)'],
98      filled=True,
99      rounded=True,
100     fontsize=10,
101     ax=ax
102 )
103 plt.title("Visualisasi Struktur Node Decision Tree (max_depth=3) - Titanic Dataset", fontsize=15, fontweight='bold', pad=15)
104 plt.tight_layout()
105 plt.savefig(png_path, bbox_inches='tight')
106 plt.close(fig)
107 print("=" * 65)
108 print(f"[OK] Gambar visualisasi node yang jelas berhasil disimpan ke:\n     {png_path}")
109
```

9. DECISION TREE VISUALIZATION



KESIMPULAN

Dari praktikum ini saya membuktikan bahwa pemilihan fitur, konsistensi imputasi data, dan kontrol kedalaman model memegang peran penting dalam menghasilkan klasifikasi yang akurat sekaligus mudah diinterpretasikan :

- Dominasi Fitur Demografis (Sex & Age): Jenis kelamin terbukti menjadi atribut penentu utama (root node) dengan tingkat kepentingan tertinggi, di mana kelompok perempuan serta anak-anak laki-laki di kelas tiket atas memiliki peluang keselamatan jauh lebih besar.
- Efektivitas Pre-Pruning (max_depth=3): Pembatasan kedalaman pohon berhasil mencegah terjadinya overfitting, menyederhanakan hierarki percabangan menjadi 8 daun (leaf nodes), dan tetap mempertahankan performa generalisasi yang baik pada data uji.
- Interpretabilitas Aturan Logika: Model Decision Tree memberikan transparansi penuh terhadap proses pengambilan keputusan melalui representasi teks aturan (tree rules) dan diagram grafis yang secara jelas memetakan ambang batas (threshold) pada setiap kondisi fitur.

TUGAS M6 - DATA MINING

TERIMA KASIH

You can access my GitHub repository for the source code :

https://github.com/GusthiPangestu1906/WDM.git